Tomato Quality Assessment with YOLOv11

A computer-vision research project for detecting fresh and spoiled tomatoes. The model is hosted through Roboflow and is intended to support fast, visual quality screening in food and agricultural supply chains.

Roboflow project: Tomatoes 9000 — model page
GitHub repository: IqraHassan-researcher/IqraHassan

> This is a research prototype, not a food-safety certification system. A visual prediction does not determine microbial safety, chemical contamination, internal quality, or Brix value.

Project overview

The project applies a YOLOv11 object-detection model to images of tomatoes, with the research goal of distinguishing fresh from spoiled produce. The work was developed as part of an MPhil thesis in Food Safety and Quality Management. Model predictions were also compared with fruit Brix measurements as a separate validation reference, and cloud-based and local-machine execution were evaluated for practical deployment.

The project page is hosted in Roboflow. Its model details may require access to the workspace. The exact published model version, class names, dataset split, and evaluation figures should be taken from the specific Roboflow version used for an experiment.

Project links

- Roboflow model page
- Roboflow workspace
- Roboflow Inference SDK documentation
- Roboflow Serverless Hosted API documentation
- Google Colab training notebook

Training notebook

The linked Colab notebook documents a YOLOv11 training workflow using Ultralytics. It expects a tomato image-and-label dataset supplied by the user; the notebook does not include the dataset archive or trained weights. Its example prepares a 90/10 train-validation split and runs `yolo11s.pt` for 60 epochs at image size 640. In Colab, select a GPU runtime as described in the notebook before training.

The notebook's example configuration is useful for understanding the training process, but it does not identify the exact published Roboflow model version or provide a version-linked held-out test report. Treat the notebook's training settings and the Roboflow-hosted model as related project materials, not as proof that they are the same model artifact.

Results and evaluation

The project author's CV describes **mAP in the high-80% range** and **precision and recall in the 80% range**. These are approximate summary figures; exact values, metric definitions, model version, evaluation split, and test-set size are not specified in the repository yet. For a reproducible benchmark, consult the evaluation page for the exact Roboflow version and record those details here before comparing results.

Evaluation item	Value
Model	YOLOv11, as described in the project CV
mAP	High-80% range, approximate CV summary; exact version-specific value not yet documented
Precision / recall	80% range, approximate CV summary; exact values not yet documented
Evaluation split and sample count	Verify and add from the selected Roboflow version
Class names	Verify from the selected Roboflow version

A useful evaluation report should identify the metric variant (for example, mAP@0.5 or mAP@0.5:0.95), the dataset split, the number of test images, and the model version. Do not compare scores from different splits or model versions as if they were measured under identical conditions.

Requirements

- Python 3.9 or newer
- A Roboflow API key with permission to access the model
- The model ID for the published Roboflow version, in the form `project-slug/version-number`

The hosted inference example below sends an image to Roboflow's Serverless Hosted API. An internet connection is required. API access and usage may be subject to Roboflow account limits and plan terms.

Installation

Clone the repository and create a virtual environment:

```bash
git clone https://github.com/IqraHassan-researcher/IqraHassan.git
cd IqraHassan
python -m venv .venv
```

Activate the environment:

```bash
# macOS / Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

Install the Roboflow Inference SDK:

```bash
python -m pip install --upgrade pip
python -m pip install inference-sdk
```

The `inference-sdk` package is an HTTP client for a Roboflow inference server; it does not itself contain this project's model weights. See the Inference SDK reference for other deployment options.

Configure access

In your Roboflow account, locate an API key and the model version ID on the project's model/deployment page. The model ID follows this pattern:

```
tomatoes-9000-nuty1/<version-number>
```

The project slug is visible in the supplied project URL, but the correct version number must be selected from the Roboflow workspace. Do not guess it.

Set the key and model ID as environment variables rather than placing credentials in source code:

Bash

```bash
# macOS / Linux
export ROBOFLOW_API_KEY="your-roboflow-api-key"
export ROBOFLOW_MODEL_ID="tomatoes-9000-nuty1/1"  # replace 1 with the version
```

Plain Text

```
# Windows PowerShell
$env:ROBOFLOW_API_KEY = "your-roboflow-api-key"
$env:ROBOFLOW_MODEL_ID = "tomatoes-9000-nuty1/1"  # replace 1 with the intended
```

Keep API keys private. Never commit them to GitHub, paste them into an issue, or embed them in a public frontend. If a key is accidentally exposed, revoke it in Roboflow and create a new one.

Run an image prediction

Save the following example as `predict.py` in the repository root. It accepts a local image path and prints the raw prediction response as JSON.

Python

```python
import json
import os
import sys
from pathlib import Path

from inference_sdk import InferenceConfiguration, InferenceHTTPClient


def main() -> None:
    if len(sys.argv) != 2:
        raise SystemExit("Usage: python predict.py path/to/image.jpg")

    image_path = Path(sys.argv[1])
    if not image_path.is_file():
        raise SystemExit(f"Image file not found: {image_path}")

    api_key = os.environ.get("ROBOFLOW_API_KEY")
    model_id = os.environ.get("ROBOFLOW_MODEL_ID")
    if not api_key:
```

```
        raise SystemExit("Set the ROBOFLOW_API_KEY environment variable first."
    if not model_id:
        raise SystemExit("Set ROBOFLOW_MODEL_ID to your Roboflow project/versio

    client = InferenceHTTPClient(
        api_url="https://serverless.roboflow.com",
        api_key=api_key,
    ).configure(InferenceConfiguration(api_key_transport="header"))

    result = client.infer(str(image_path), model_id=model_id)
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```

Run it with an image of your choice:

Bash

```
python predict.py path/to/tomato.jpg
```

The response contains the predictions returned by the selected Roboflow model version. Field names and available output can depend on the model and inference configuration. Review the response and the version's class names rather than assuming labels or thresholds.

Demo images and video

Use images and video that you own or are permitted to share. Strong examples show the original frame alongside the actual model prediction, with the model version identified. If you add annotated examples to this repository, include a short caption describing the class, confidence threshold, and source of the prediction. Do not present hand-drawn boxes, mock predictions, or unrelated tomato imagery as outputs from this model.

For video inference or streaming, consult Roboflow's current Inference SDK documentation and the relevant model deployment instructions. The project-specific video workflow has not been documented in this repository yet.

Research notes and limitations

- The model predicts visual patterns represented in its training data; performance can vary with lighting, camera, cultivar, occlusion, ripeness, and image quality.

- The CV describes a comparison between cloud and local execution and reports that local execution was more efficient for real-time use. Hardware, latency measurements, and test procedure are not included here, so this should be treated as a research observation rather than a general performance guarantee.
- Brix measurement was used as a validation reference in the thesis work. A visual model prediction is not a direct Brix measurement.
- The model's predictions are not a substitute for food-safety testing, expert inspection, or regulatory procedures.

Data, model access, and licensing

The dataset and trained model are linked through the Roboflow project. Access, download permissions, model availability, and reuse rights are governed by the settings and license shown in that project. No separate dataset or model license is specified in this repository. Please verify the applicable terms before redistributing data, weights, or prediction examples.

Author

Iqra Hassan — AI vision researcher in food safety and quality management.
Email: officialiqrahassan@gmail.com
GitHub: @IqraHassan-researcher

References

- Roboflow Inference SDK
- Roboflow Serverless Hosted API
- Roboflow project and model page